

IIITS

Computer Vision Autonomous Patrol Simulator

Preliminary Project Report

Computer Vision and Explainable Navigation

Author: Charanpreet Singh

Status: Student

Student ID: OMIO25S00012

Affiliation: IIITS

Submission date: 4 October 2026

Document status: Preliminary. The implementation and tests described below are complete as of 3 October 2026; real-footage evaluation and final packaging are scheduled before submission.

Abstract—Camera-based object detection identifies obstacles but does not directly explain how a navigation system should respond. This project develops a software-only patrol simulator that converts webcam or video frames into an inspectable straight, turn, or stop recommendation. The current prototype combines an optional Ultralytics YOLO11n detector, a class-aware intersection-over-union tracker, horizontal navigation zones, an image-space collision corridor, approximate monocular distance bands, per-object risk scores, and left/right corridor aggregation. A PySide6 dashboard presents annotated frames, the selected action and reason, steering angle, simulated wheel speeds, risk map, and telemetry. The implementation is modular and configurable through YAML. Initial verification on 3 October 2026 found that all six headless navigation tests pass; the installed model loads on CPU; and the 24-second synthetic demonstration video yields no YOLO detections at five sampled times. Controlled injected-detection scenes nevertheless exercise the downstream turn and stop logic and are labeled as such. These results establish a working decision prototype, not detection accuracy or safe autonomous operation. Before the 4 October 2026 submission, the remaining work is to run a small real-footage check, document observed inference behavior and limitations, verify the final demonstration, and package the code and report. Future improvements include camera calibration, stronger tracking, and consistency fixes for corridor display and stop handling.

Index Terms—computer vision, object detection, monocular distance, risk assessment, explainable navigation, simulation

I. INTRODUCTION

A. Background and Purpose

Detecting a person or vehicle is only the first step in visual navigation. A camera-based application must also decide whether the object lies near a projected route, whether another route appears less occupied, and when to stop. General object-detection datasets such as COCO address recognition and localization in complex scenes [2]; they do not, by themselves, define a safe navigation policy. This project therefore focuses on the transparent transformation from image observations to a *simulated* decision. It does not drive hardware.

The operational goal is to accept webcam or recorded video, detect configured object classes when a model is available, attach stable identities to detections, estimate image-space position and rough proximity, combine scene risks, and display the resulting recommendation together with its reason. The user should be able to replay a scenario, inspect the intermediate overlays, and compare the left and right corridor scores.

B. Objectives and Boundaries

The specific objectives are to (1) build a repeatable capture and detection path, (2) associate detections across frames, (3) evaluate approximate proximity and navigation zones, (4) choose a lower-risk route or stop based on multiple objects, and (5) expose each stage in a desktop dashboard and log. The boundary is important: a single uncalibrated RGB camera does not supply reliable metric depth, and the wheel values are display outputs, not commands to an actuator. No mapping, localization, or roadworthiness claim is made.

II. PROGRESS TO DATE AND METHODOLOGY

A. Implemented Pipeline

The application currently uses OpenCV capture for a webcam or MP4 file. A configurable Ultralytics YOLO11n model [1] accepts each frame with default confidence and non-maximum-suppression IoU parameters of 0.45 and an inference size of 640 pixels. The adapter retains the configured classes: person, bicycle, car, motorcycle, bus, truck, and chair. If the model is disabled or cannot load, the interface remains usable but returns no detections. The detector is based on a COCO-pretrained model according to the vendor documentation [1]; no project-specific model training has been performed.

The custom tracker matches same-class detections to existing tracks by bounding-box IoU, with a default matching threshold of 0.25 and a 12-frame missing-track allowance. Its area history labels apparent approach or retreat. This tracker is intentionally simpler than published tracking-by-detection systems such as SORT, which additionally use prediction and assignment machinery [3]; SORT performance numbers are not attributed to this implementation.

Each detection is assigned to one of five equally spaced horizontal zones by its box center. A separate central image-space corridor test increases the risk of objects whose centers fall in the projected path. Distance is approximated by

$$\hat{d} = \frac{f_{\text{px}} H_{\text{prior}}}{h_{\text{box}}}, \quad (1)$$

where f_{px} defaults to 700 pixels, H_{prior} is an assumed height for the detected class, and h_{box} is bounding-box height. The result is categorized as safe, caution, near, or critical using the configured thresholds of 6, 4, and 2 m. This pinhole-style relationship depends on camera calibration and object geometry [4]; the displayed value is explicitly labeled an estimate.

The heuristic per-object risk combines proximity, normalized box area, lower-frame position, corridor membership, class weight, confidence, and an approach bonus. The highest risk in each zone forms a zone score. Left and right zone scores are aggregated with a noisy-OR expression, $R_{\text{side}} = 100[1 - \prod_j (1 - R_j/100)]$. The decision engine requests straight motion below risk 42, selects the lower-risk side when one side remains below 78, and otherwise requests a stop. Steering is smoothed before conversion to two differential-drive percentage values. The PySide6 dashboard [5] shows the annotated frame, explanation, gauges, state history, and CSV telemetry.

B. Data and Investigation Procedure

The available repeatable input is a 24-second synthetic MP4 generated by the repository. It contains clear-path, approaching-center, side-obstacle, and blocked-path drawings. It is used to test playback, rendering, and timing rather than detector accuracy. A separate controlled capture script injects known detection boxes over this video to verify the downstream tracker, risk, decision, and GUI path. No labeled

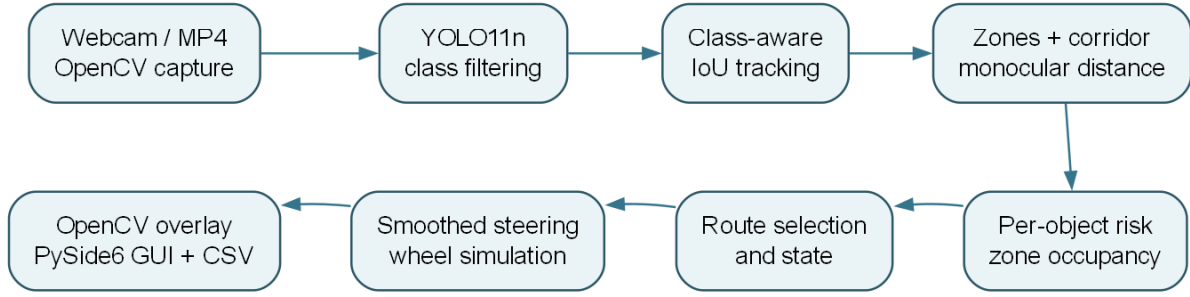


Fig. 1. Implemented perception-to-decision path. The wheel values and trajectory are simulations. Diagram generated from the project’s module structure.

real-footage evaluation set, survey, or calibrated range measurements has yet been gathered.

The investigation has combined source review, deterministic unit tests, sampling of the synthetic video with the installed model, and inspection of actual Qt dashboard captures. Tests exercise an empty scene, lower-risk side selection, blocked-side emergency classification, zone boundaries, wheel direction, and steering clamp. The repository report and proposal record the full implementation and screenshot provenance [6].

C. Preliminary Results

On 3 October 2026, `python -m pytest -q` passed all six existing navigation tests. The included `yolo11n.pt` model loaded with CPU status. At 2, 6, 11, 16, and 21 seconds in the synthetic MP4, the detector returned zero objects. This finding limits the video to a GUI and logic fixture; it cannot demonstrate YOLO recognition performance. In contrast, controlled injected boxes produced the expected left-turn and blocked-route stop displays through the normal downstream pipeline. The displayed stop occurs immediately in that capture because the capture script sets the action hold to zero; normal configuration uses a 0.35-second minimum action duration.

III. PLANNED WORK AND CALENDAR TIMELINE

The final project deadline supplied for this report is 4 October 2026. The short remaining window calls for focused validation and clear reporting, rather than a new model-training effort. Table I separates completed work from the immediate submission tasks.

If suitable real footage cannot be obtained in time, the final submission will label that evaluation as incomplete rather than substitute synthetic detections. The proposed success criteria are a reproducible launch, passing existing tests, correct decision behavior for controlled scenes, and an honest account of inference behavior and limitations. Larger revisions such as calibrated depth or a worker thread for inference remain future work.

IV. CURRENT RISKS AND LIMITATIONS

The principal technical uncertainty is the monocular distance estimate. Unknown real object height, partial boxes,

TABLE I
SCHEDULE TO FINAL SUBMISSION

Date	Task	Evidence
3 Oct 2026	Freeze implementation baseline; run navigation tests; compile preliminary report.	Six passing tests and PDF draft.
4 Oct 2026 morning	Try YOLO-enabled inference on available real footage; record classes, misses, model status, and frame timing.	Short observation table with input and settings.
4 Oct 2026 afternoon	Rehearse clear, turn, and blocked demos; check labels and known limitations; submit source and final documents.	Reproducible demo and packaged report.

camera tilt, and lens properties can change the estimate; the project has not calibrated the camera. The displayed trapezoidal corridor differs from the center-rectangle geometry used in analysis. An inference exception currently yields an empty detection list, which can resemble a clear scene. The action hold may delay a requested stop, while the manual emergency override zeros the simulated wheels on the next processed frame without synchronizing the state history. Finally, the GUI timer runs inference in the GUI thread, so slow inference can reduce responsiveness. These findings define the interpretation of the preliminary results and the priority of future fixes.

V. CONCLUSION

The implemented prototype already demonstrates the complete capture-to-explanation software chain and passes the six available navigation tests. Its strongest contribution is visibility into how detection, approximate scene geometry, and configurable risk rules produce a simulated steering choice. The remaining work before the stated deadline is a small real-footage check, final demonstration verification, and careful packaging. The report does not claim validated detection accuracy, measured depth, or safe physical autonomy.

REFERENCES

- [1] G. Jocher and J. Qiu, “Ultralytics YOLO11,” Ultralytics, 2024. [Online]. Available: <https://docs.ultralytics.com/models/yolo11/>. Accessed: Oct. 3, 2026.

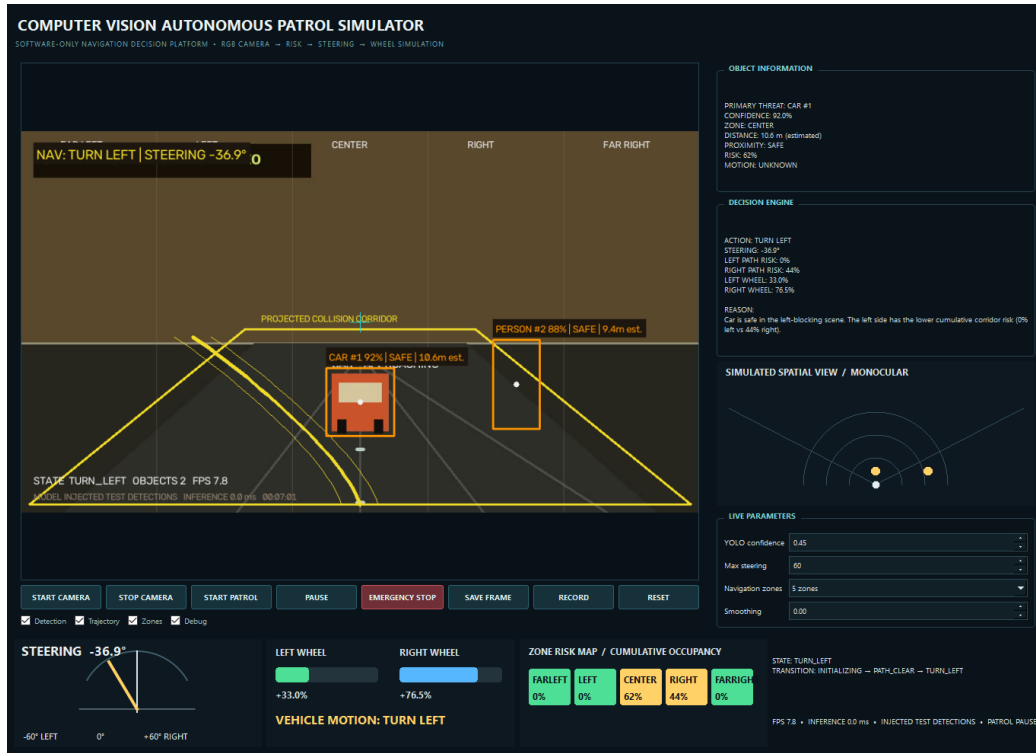


Fig. 2. Actual dashboard capture of a controlled left-turn case. Detection boxes were injected over synthetic video; this is evidence of navigation logic and visualization, not YOLO detection accuracy.

- [2] T.-Y. Lin *et al.*, “Microsoft COCO: Common objects in context,” in *Proc. European Conf. Computer Vision*, 2014, pp. 740–755. [Online]. Available: <https://arxiv.org/abs/1405.0312>.
- [3] A. Bewley, Z. Ge, L. Ott, F. Ramos, and B. Upcroft, “Simple online and realtime tracking,” in *Proc. IEEE Int. Conf. Image Processing*, 2016, pp. 3464–3468. [Online]. Available: <https://arxiv.org/abs/1602.00763>.
- [4] OpenCV, “Camera calibration,” OpenCV documentation. [Online]. Available: https://docs.opencv.org/5.0/main_modules/calib.html. Accessed: Oct. 3, 2026.
- [5] The Qt Company, “Qt for Python,” Qt documentation. [Online]. Available: <https://doc.qt.io/qtforpython-6/>. Accessed: Oct. 3, 2026.
- [6] C. Singh, “Computer Vision Autonomous Patrol Simulator: Engineering analysis and demonstration report,” project repository, Oct. 2026, docs/PROJECT_REPORT.md.

APPENDIX A

SUPPLEMENTARY TECHNICAL DOCUMENTS

The repository provides README.md for installation and operation, config.yaml for default parameters, docs/PROJECT_REPORT.md for detailed code analysis, docs/PROJECT_PROPOSAL.md for the presentation concept, tests/test_navigation.py for the six unit tests, and docs/assets/ for the architecture and GUI figures. The synthetic test video is generated by tools/create_sample_video.py. No preliminary human survey was performed.

APPENDIX B

REPRODUCTION COMMANDS

From the repository root, install requirements.txt, run `python -m pytest -q`, generate the sample with `python tools/create_sample_video.py`,

and launch `python main.py --video recordings/sample_patrol.mp4 --no-model`. Use a real video or webcam with the model enabled to investigate detection. Run `python docs/build_preliminary_report.py` to regenerate this PDF from its $\text{\LaTeX}$ source.